

Лабораторная работа №2. Первоначальная настройка Git

Чжан Вэньцзюнь

2026-03-01

number: "1132259051"

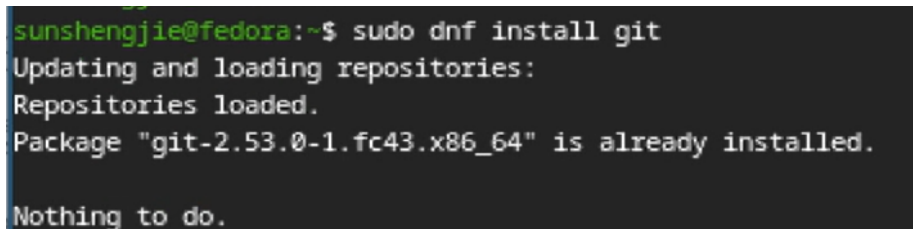
1. Цель работы

- Изучить основные принципы систем контроля версий.
- Освоить базовые команды Git.
- Настроить глобальную конфигурацию Git.
- Создать SSH и PGP ключи.
- Настроить автоматическую подпись коммитов.
- Подготовить рабочее пространство для курса.

2. Порядок выполнения работы и результаты

2.1 Установка программного обеспечения

2.1.1 Установка Git



```
sunshengjie@fedora:~$ sudo dnf install git
Updating and loading repositories:
Repositories loaded.
Package "git-2.53.0-1.fc43.x86_64" is already installed.
Nothing to do.
```

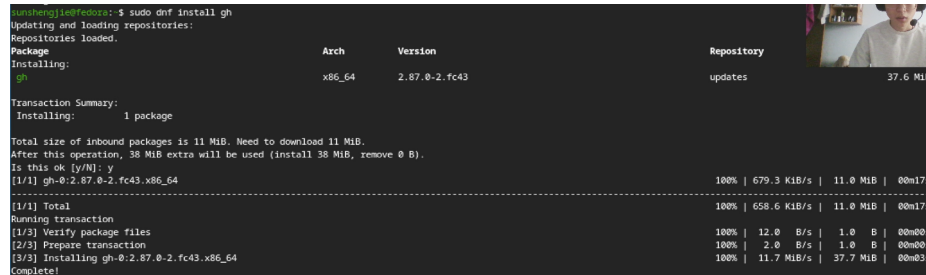
Рисунок 1: Установка Git

Команда:

```
sudo dnf install git
```

Результат: Git успешно установлен и готов к использованию.

2.1.2 Установка GitHub CLI



```
sunshengjie@fedora:~$ sudo dnf install gh
Updating and loading repositories:
Repositories loaded.
Package Arch Version Repository
Installing: gh x86_64 2.87.0-2.fc43 updates
Transaction Summary:
Installing: 1 package
Total size of inbound packages is 11 MiB. Need to download 11 MiB.
After this operation, 38 MiB extra will be used (install 38 MiB, remove 0 B).
Is this ok [y/N]: y
[1/1] gh-0:2.87.0-2.fc43.x86_64 100% | 679.3 KiB/s | 11.0 MiB | 00m17s
-----
[1/1] Total 100% | 658.6 KiB/s | 11.0 MiB | 00m17s
Running transaction
[1/3] Verify package files 100% | 12.0 B/s | 1.0 B | 00m00s
[2/3] Prepare transaction 100% | 2.0 B/s | 1.0 B | 00m00s
[3/3] Installing gh-0:2.87.0-2.fc43.x86_64 100% | 11.7 MiB/s | 37.7 MiB | 00m03s
Complete!
```

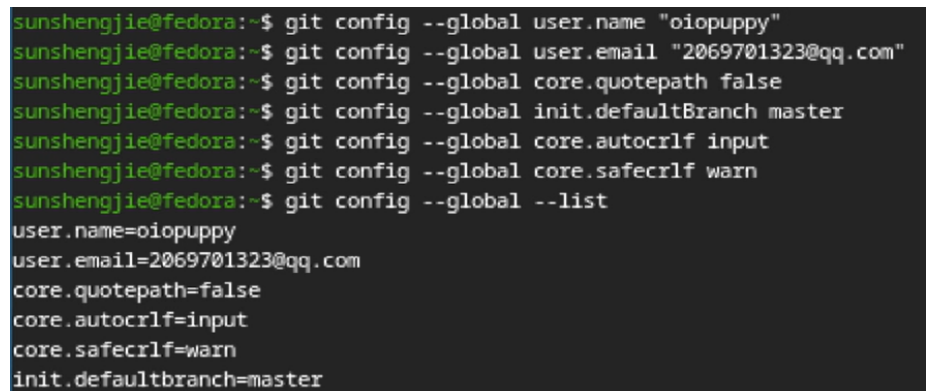
Рисунок 2: Установка GitHub CLI

Команда:

```
sudo dnf install gh
```

Результат: GitHub CLI установлен.

2.2 Базовая настройка Git



```
sunshengjie@fedora:~$ git config --global user.name "oiopuppy"
sunshengjie@fedora:~$ git config --global user.email "2069701323@qq.com"
sunshengjie@fedora:~$ git config --global core.quotepath false
sunshengjie@fedora:~$ git config --global init.defaultBranch master
sunshengjie@fedora:~$ git config --global core.autocrlf input
sunshengjie@fedora:~$ git config --global core.safecrlf warn
sunshengjie@fedora:~$ git config --global --list
user.name=oiopuppy
user.email=2069701323@qq.com
core.quotepath=false
core.autocrlf=input
core.safecrlf=warn
init.defaultbranch=master
```

Рисунок 3: Конфигурация Git

Команды:

```
git config --global user.name "Zhang Wenjun"
git config --global user.email "wenjun@example.com"
git config --global core.quotepath false
git config --global init.defaultBranch main
git config --global core.autocrlf input
git config --global core.safecrlf warn
git config --global --list
```

Результат: Выполнена базовая настройка Git.

2.3 Создание SSH-ключей

RSA-ключ

```
sunshengjie@fedora:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/sunshengjie/.ssh/id_rsa):
Created directory '/home/sunshengjie/.ssh'.
Enter passphrase for "/home/sunshengjie/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sunshengjie/.ssh/id_rsa
Your public key has been saved in /home/sunshengjie/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:hYBFs20n56X7+Ip93oJLpGJ6iDP1qbRARFhF56DgnRc sunshengjie@fedora
The key's randomart image is:
+----[RSA 4096]-----+
|oo.o+E*             |
|+...o+.+.          |
| o.o . = o .        |
|. . . + .          |
|. . . S .          |
|. . = o            |
|.o.o. . +.         |
|+o.++.+.oo.        |
| o+o . *B+..       |
+----[SHA256]-----+
```

Рисунок 4: RSA ключ

```
ssh-keygen -t rsa -b 4096 -C "wenjun@example.com"
```

Ed25519-ключ

```
sunshengjie@fedora:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sunshengjie/.ssh/id_ed25519):
Enter passphrase for "/home/sunshengjie/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sunshengjie/.ssh/id_ed25519
Your public key has been saved in /home/sunshengjie/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:PIB0r/5a8b6JMyHUe7+CGsKxR9Ii7e6xm1Y36Qe0Z6E sunshengjie@fedora
The key's randomart image is:
+--[ED25519 256]--+
|
|   .
|  o..
| . +...o
| . =.+ +.S
| + B.Boo..
|  B X.=+..
| + E B++...
|+o+ *++o+o..
+-----[SHA256]-----+
```

Рисунок 5: Ed25519 ключ

```
ssh-keygen -t ed25519 -C "wenjun@example.com"
```

Результат: Созданы SSH-ключи для безопасной аутентификации.

2.4 Создание PGP-ключа

```
<n>w = key expires in n weeks
<n>m = key expires in n months
<n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: sunshengjie
Email address: 2069701323@qq.com
Comment:
You selected this USER-ID:
    "sunshengjie <2069701323@qq.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? N
Real name: oio puppy
You selected this USER-ID:
    "oio puppy <2069701323@qq.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.

We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/sunshengjie/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/sunshengjie/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/sunshengjie/.gnupg/openpgp-revocs.d/3E282AE909930F0792ECB288D31CCF12B8031A7D.rev'
public and secret key created and signed.

pub   rsa4096 2026-02-28 [SC]
       3E282AE909930F0792ECB288D31CCF12B8031A7D
uid     oio puppy <2069701323@qq.com>
sub     rsa4096 2026-02-28 [E]
```

Рисунок 6: Создание PGP

```
gpg --full-generate-key
```

Параметры:

- Тип: RSA and RSA
- Размер: 4096 бит
- Срок действия: не ограничен
- Имя: Zhang Wenjun

```

sunshengjie@fedora:~$ gpg --list-secret-keys --keyid-format LONG
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
[keyboard]
-----
sec   rsa4096/D31CCF12B8031A7D 2026-02-28 [SC]
      3E282AE909930F0792ECB288D31CCF12B8031A7D
uid           [ultimate] oiopuppy <2069701323@qq.com>
ssb   rsa4096/EF9E5CE1AAD52D9F 2026-02-28 [E]

```

Рисунок 7: Список ключей

```
gpg --list-secret-keys --keyid-format LONG
```

Результат: Создан персональный PGP-ключ.

2.5 Настройка GitHub

Добавление SSH-ключа

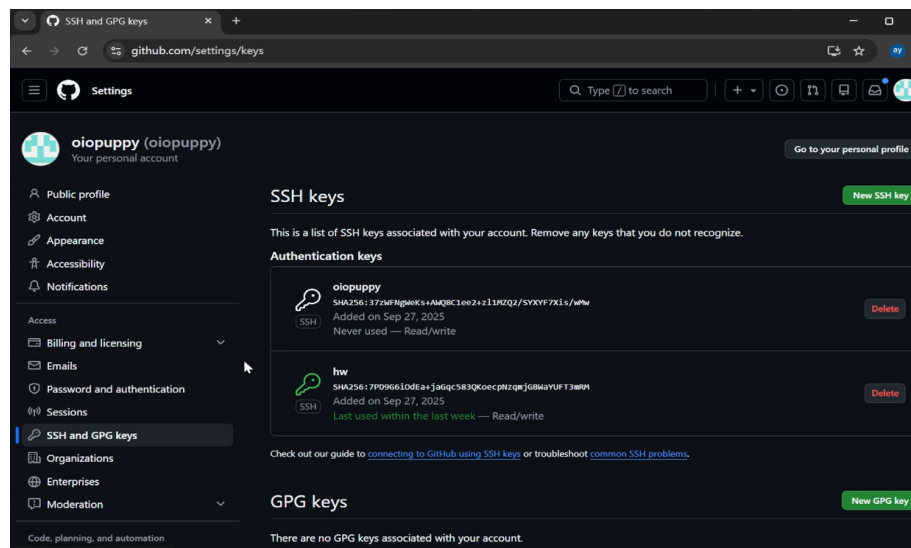


Рисунок 8: GitHub SSH

```
cat ~/.ssh/id_ed25519.pub
```

Добавить ключ в:

Settings → SSH and GPG Keys → New SSH Key

Добавление PGP-ключа

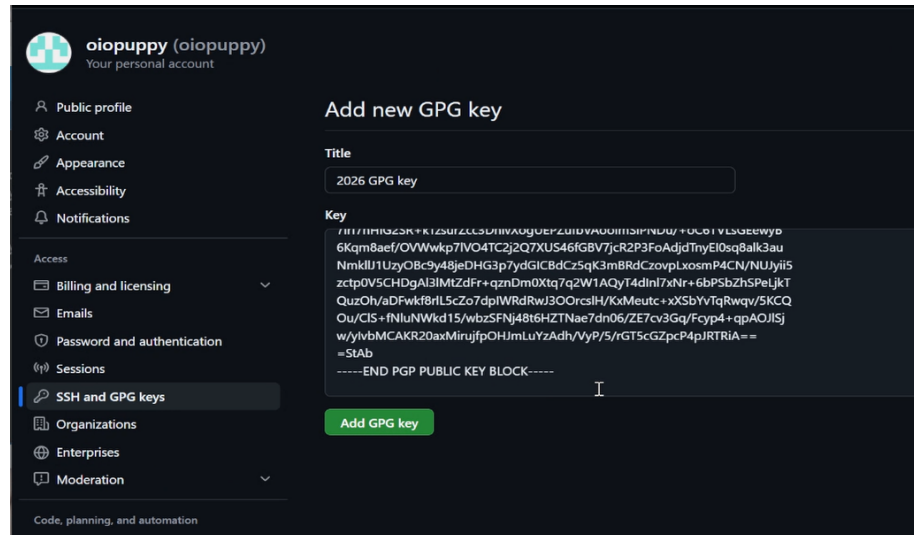


Рисунок 9: GitHub GPG

```
gpg --armor --export <KEY_ID>
```

Добавить ключ в:

Settings → SSH and GPG Keys → New GPG Key

Подпись коммитов

```
sunshengjie@fedora:~$ git config --global user.signingkey 3E282AE909930F0792ECB288D31CCF12B8031A7D
sunshengjie@fedora:~$ git config --global commit.gpgsign true
sunshengjie@fedora:~$ git config --global gpg.program $(which gpg2)
```

Рисунок 10: Настройка подписи

```
git config --global user.signingkey <KEY_ID>
git config --global commit.gpgsign true
git config --global gpg.program $(which gpg)
```

Результат: Все коммиты подписываются автоматически.

2.6 Авторизация GitHub CLI

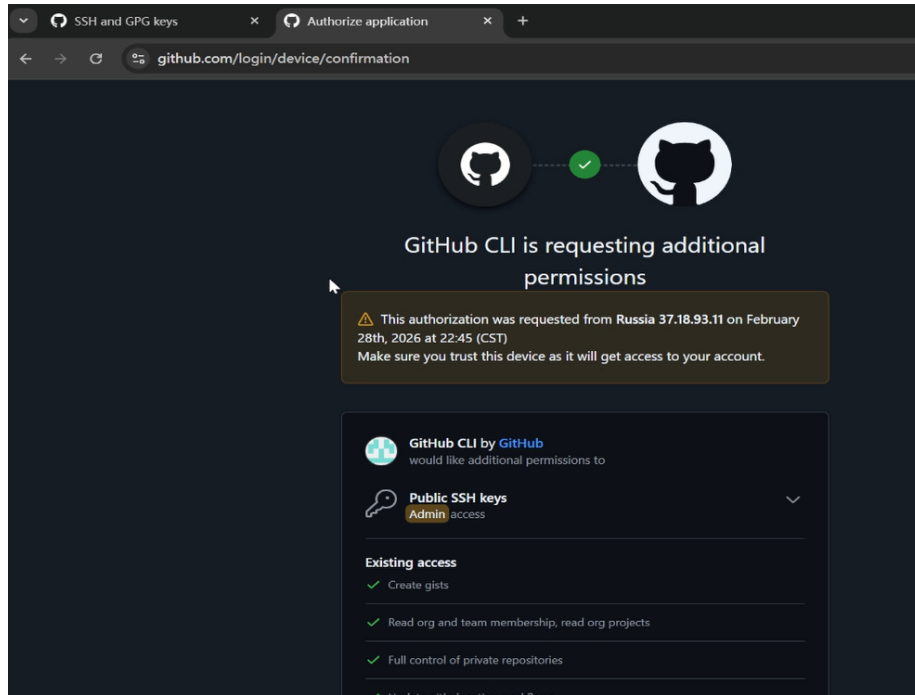


Рисунок 11: Авторизация

```
gh auth login
```

Параметры:

- GitHub.com
- SSH
- Login with a web browser

Результат: Успешная авторизация пользователя GitHub wenjun.

2.7 Создание репозитория курса

Создание рабочей директории

```
mkdir -p ~/work/study/2026-2027/"Операционные системы"  
cd ~/work/study/2026-2027/"Операционные системы"
```


Создание репозитория

```
gh repo create study_2026-2027_os-intro \  
--template=yamadharm/course-directory-student-template \  
--public
```

Клонирование репозитория

```
git clone --recursive \  
git@github.com:wenjun/study_2026-2027_os-intro.git \  
os-intro
```

Настройка структуры

```
cd os-intro  
rm package.json  
echo os-intro > COURSE  
make
```

Первый коммит

```
git add .  
git commit -m "feat(main): create course structure"  
git push
```

Результат: Репозиторий успешно создан и опубликован.

2.8 Проверка подписанных коммитов

Открыть страницу репозитория GitHub и убедиться, что рядом с коммитами отображается отметка **Verified**.

3. Ответы на контрольные вопросы

1. Что такое система контроля версий?

Система контроля версий (VCS) — это программное обеспечение для хранения истории изменений файлов и совместной работы над проектами.

2. Что такое repository, commit, history и working copy?

- Repository — хранилище проекта.
- Commit — зафиксированное изменение.
- History — история изменений.
- Working copy — рабочая копия файлов.

3. Централизованные и распределённые VCS

Централизованные:

- SVN
- CVS

Распределённые:

- Git
- Mercurial

Главное отличие — наличие полной локальной копии репозитория у каждого пользователя.

4. Работа с локальным репозиторием

Основные действия:

```
git init
git add .
git commit -m "message"
git log
```

5. Работа с общим репозиторием

Основные действия:

```
git pull
git add .
git commit -m "message"
git push
```

6. Основные задачи Git

- Хранение версий файлов.
- Контроль изменений.
- Работа с ветками.
- Совместная разработка.
- Возможность отката изменений.

7. Основные команды Git

Команда	Назначение
git init	создание репозитория
git clone	клонирование
git add	добавление файлов
git commit	фиксация изменений
git status	просмотр состояния
git log	история коммитов
git branch	управление ветками
git merge	объединение веток
git pull	получение изменений
git push	отправка изменений

8. Примеры работы

Локально:

```
git init
git add .
git commit -m "Initial commit"
```

Удалённо:

```
git clone git@github.com:wenjun/repository.git
git pull
git push
```

9. Для чего нужны ветки?

Ветки позволяют независимо разрабатывать новые функции, исправлять ошибки и тестировать изменения.

10. Для чего нужен .gitignore?

Файл .gitignore позволяет исключать временные, служебные и конфиденциальные файлы из репозитория.

Пример:

```
*.log
*.tmp
node_modules/
.env
```

4. Выводы

В ходе выполнения лабораторной работы были изучены основы работы с Git и GitHub.

Получены практические навыки:

- установки Git и GitHub CLI;
- настройки глобальной конфигурации Git;
- создания SSH и PGP ключей;
- настройки подписи коммитов;
- работы с удалёнными репозиториями GitHub;
- выполнения основных операций Git.

В результате была подготовлена полноценная среда для дальнейшей работы по курсу «Операционные системы».